



CSS Responsive Web Design (Part 2)

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
Faculty of Engineering
Khon Kaen University





Agenda



- Images
- CSS Flexbox
- Alternatives to Responsive Web Design



Responsive images



- A study found out that 50% of a webpage's size is that of images
- Images that are not responsive can mess up the whole alignment of the website
- Non-responsive images are also unable to deliver the meaning on the mobile view of the website



Using the Width Property



If the **width** property is set to a percentage and the height is set to "auto", the image will be responsive and scale up and down

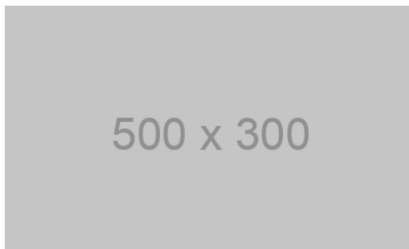
```
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
img {
  width: 100%;
  height: auto;
}
</style>
</head>
```



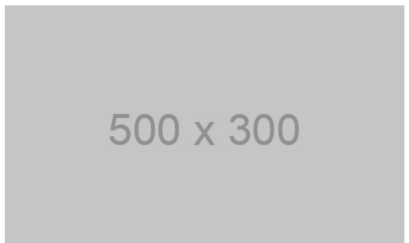
Example of Using width Property



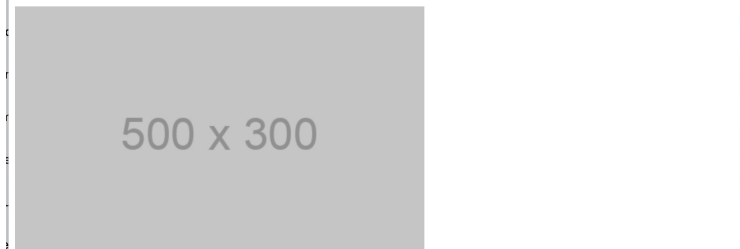
Non-responsive Image



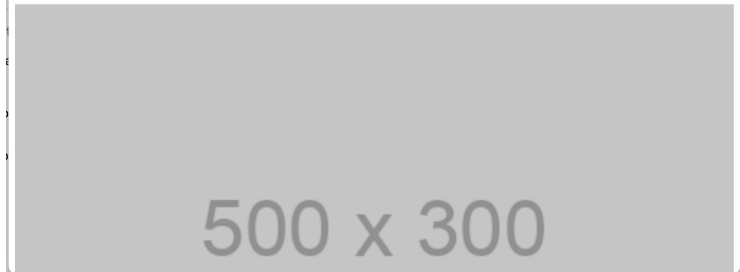
Responsive Image (Width Property)



Non-responsive Image



Responsive Image (Width Property)



responsive > <> width-property-comparison.html > html > body >

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width,
6       initial-scale=1.0" />
7     <title>Width Property Comparison</title>
8   </head>
9   <body>
10    <h2>Non-responsive Image</h2>
11    
13    <h2>Responsive Image (Width Property)</h2>
14    
19  </body>
20 </html>
```



Using the max-width Property

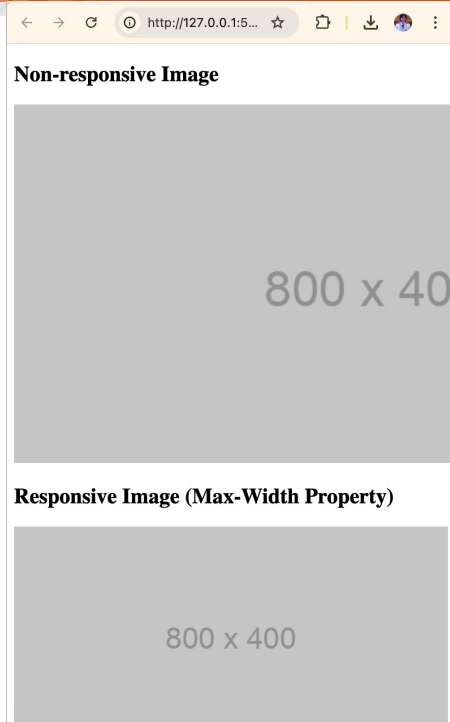


If the `max-width` property is set to 100%, the image will scale down if it has to, but never scale up to be larger than its original size

```
img {  
    max-width: 100%;  
    height: auto;  
}
```



Example of Using max-width Property



```
responsive > <> max-width-property-comparison.html > html > body > img
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width,
6        initial-scale=1.0" />
7      <title>Max-Width Property Comparison</title>
8    </head>
9    <body>
10     <h2>Non-responsive Image</h2>
11     
13     <h2>Responsive Image (Max-Width Property)</h2>
14     
19   </body>
20 </html>
```



Using width vs. max-width



- Use **width: 100%** when you always want the image to fill its container, regardless of the original image size.
- Use **max-width: 100%** as a general rule for most images. This allows the image to scale down on smaller screens, but prevents it from stretching beyond its original size on larger screens, maintaining image quality.



Background Images



- If the background-size property is set to "cover", the background image will scale to cover the entire content area
- Notice that the "cover" value keeps the aspect ratio, and some part of the background image may be clipped

```
div {  
  width: 100%;  
  height: 400px;  
  background-image: url('img_flowers.jpg');  
  background-size: cover;  
  border: 1px solid red;  
}
```

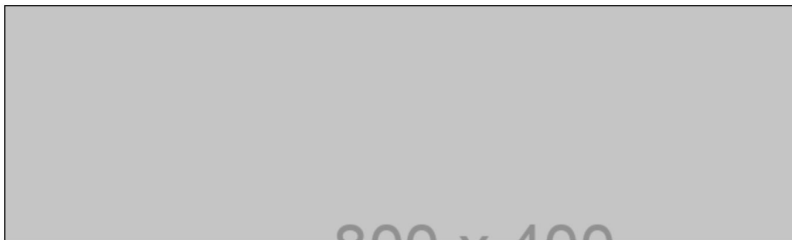


Example of Using cover for background image

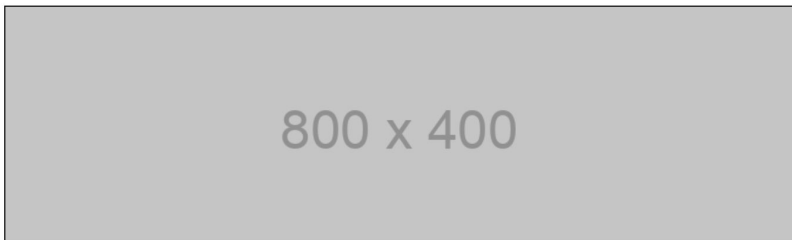


← → ↻ ⓘ http://127.0.0.1:5501/responsive/b... ☆ 🧑 📊 🗂️ ⬇️ 👤 ⋮

Non-responsive Background Image



Responsive Background Image



responsive > <> background-image-comparison.html > 📁 html > 📁 head > 📁 style

```
7   <style>
8     .bg-image {
9       height: 200px;
10      border: 1px solid #000;
11    }
12    .non-responsive {
13      background-image: url("https://via.placeholder.com/800x400");
14    }
15    .responsive {
16      background-image: url("https://via.placeholder.com/800x400");
17      background-size: cover;
18      background-position: center;
19    }
20  </style>
21  </head>
22  <body>
23    <h2>Non-responsive Background Image</h2>
24    <div class="bg-image non-responsive"></div>
25    <h2>Responsive Background Image</h2>
26    <div class="bg-image responsive"></div>
27  </body>
```



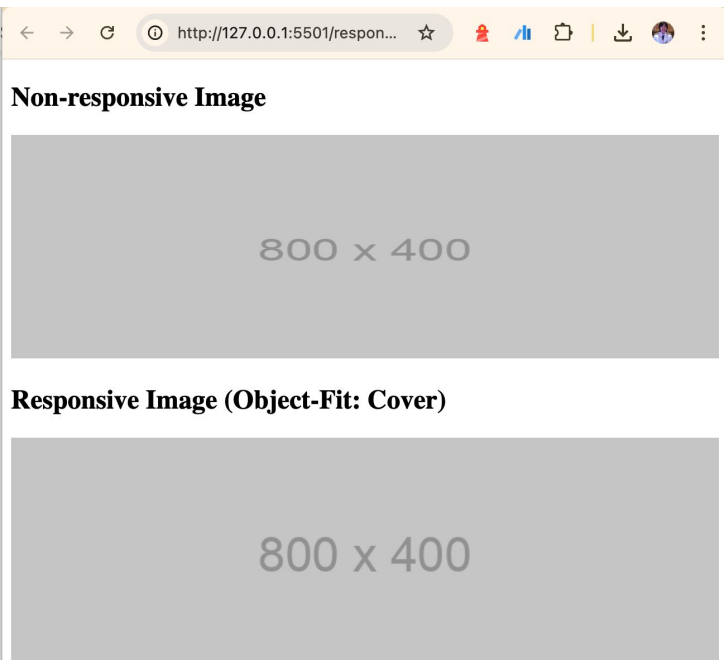
Object-Fit and Object-Position Properties



- The first property that will help us achieving this aim is object-fit property
- This property specifies how the image should resize itself when it is rendered onto different screen sizes
- Using object-fit property we can define whether we want to stretch our image in case of bigger screen sizes or shrink it for smaller ones



Sample Usage of object-fit



responsive > <> object-fit-comparison.html > html > body > img

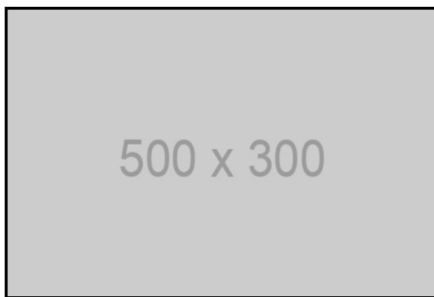
```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width,
6          initial-scale=1.0">
7      <title>Object-Fit Comparison</title>
8      <style>
9          |   img { width: 100%; height: 200px; }
10     </style>
11 </head>
12 <body>
13     |   <h2>Non-responsive Image</h2>
14     |   
16     |   <h2>Responsive Image (Object-Fit: Cover)</h2>
17     |   
20 </div>
21 </body>
22 </html>
```



Object-fit Values: Fill, Contain, Cover



Fill



Use Case: Full-width background images

Fill stretches the image to fit the container, which can distort the aspect ratio. It's useful when you need the image to cover the entire area without any empty space, regardless of distortion.

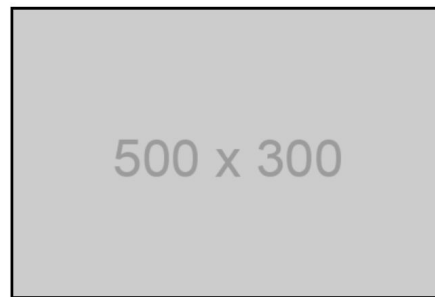
Contain



Use Case: Product images in e-commerce

Contain ensures the entire image is visible without cropping, maintaining its aspect ratio. It's ideal for showcasing products where seeing the whole item is crucial, even if it leaves empty space.

Cover



Use Case: Hero images or thumbnails

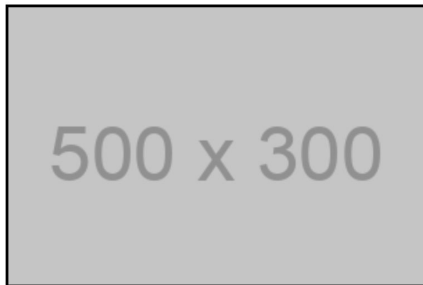
Cover fills the container while maintaining aspect ratio, cropping if necessary. It's perfect for hero images or thumbnails where you want to fill the space without distortion, accepting that some parts may be cropped.



Object-fit Values: None and Scale-down



None



Use Case: Cropped views or image sliders

None keeps the image at its original size, potentially cropping it. This is useful for creating cropped views of larger images or in image sliders where you want to show different parts of an image.

Scale-down



Use Case: User-uploaded content

Scale-down behaves like 'none' or 'contain', whichever results in a smaller image. It's great for user-uploaded content where you want to display the full image but prevent small images from being stretched.



Remove the Image Baseline Gap with `display:block`



- `<img>` is an **inline** element by default, so it sits on the text baseline
- This leaves a few pixels of blank space **below** the image inside its box, even with `height` and `object-fit` set
- The gap breaks tidy tile heights and makes rows look uneven
- Fix: set the image to block-level so it no longer aligns to the baseline



Example of the Usage of `display:block` of `img`



```
.tile img {  
  width: 100%;  
  height: 200px;  
  object-fit: cover;  
  display: block; /* removes the inline baseline gap */  
}
```

- `display: block` also lets the image fill its box cleanly with no stray whitespace beneath it
- Rule of thumb: for images inside a sized container (cards, tiles, galleries), always set `display: block`



The Usage of object-position



- Another important property used in CSS to achieve responsiveness is object-position
- This is used in addition to object-fit and often together
- The object-position property takes two values **x** and **y** which determines that the image should be placed **x** px from the left and **y** px from the top
- Here, we want the image to be moved 10px from the left and 50px from the top



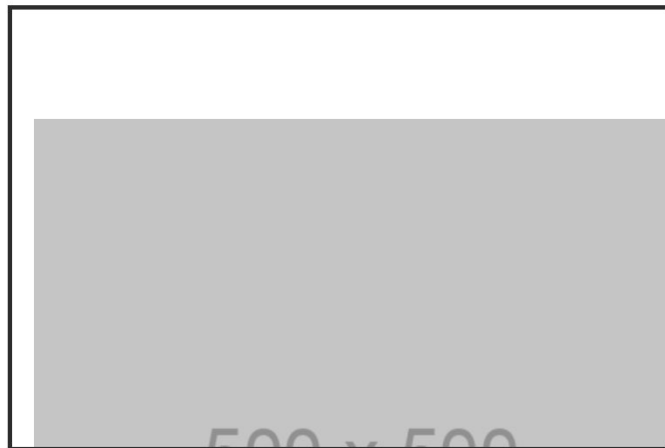
Sample usage of object-position



ve > <> responsive-image-example.html > html > head > style > .responsive-ima

```
7      <style>
8          .image-container {
9              width: 300px;
10             height: 200px;
11             border: 2px solid #333;
12             margin-bottom: 20px;
13         }
14         .responsive-image {
15             width: 100%;
16             height: 100%;
17             object-fit: cover;
18             object-position: 10px 50px; /* 10px from left, 50px from
19                                     top */
20         }
21     </style>
22 </head>
23 <body>
24     <div class="image-container">
25         
```

← → ↻ ⓘ http://127.0.0.1:5501/responsive/respo





The Values of object-position



object-position: `<x> <y>;` accepts three kinds of values:

- **Keywords:** `left | center | right (x), top | center | bottom (y)`
- **Percentages:** `0%–100%` — the most common for responsive images
- **Lengths:** `px, em, ... (e.g. 10px 50px)`

Default is `50% 50%` (center).



How Percentage Works



How percentages work:

- X% aligns the X% point of the *image* with the X% point of the *box* — not "X px from the left."
- So a lower Y% shows more of the **top** of the image, a higher Y% shows more of the **bottom**.
 - 50% 20% → horizontally centered, showing nearer the top
 - 50% 80% → horizontally centered, showing nearer the bottom



Fixing portrait crop



Problem: `object-fit: cover` crops from the center by default, so a tall portrait can lose its subject (e.g. a face at the top).

```
.tile img { width: 100%; height: 200px; object-fit: cover; /* center-crops by default */ }  
.tile-portrait img { object-position: 50% 20%; /* shift crop up to keep subject in frame */ }
```

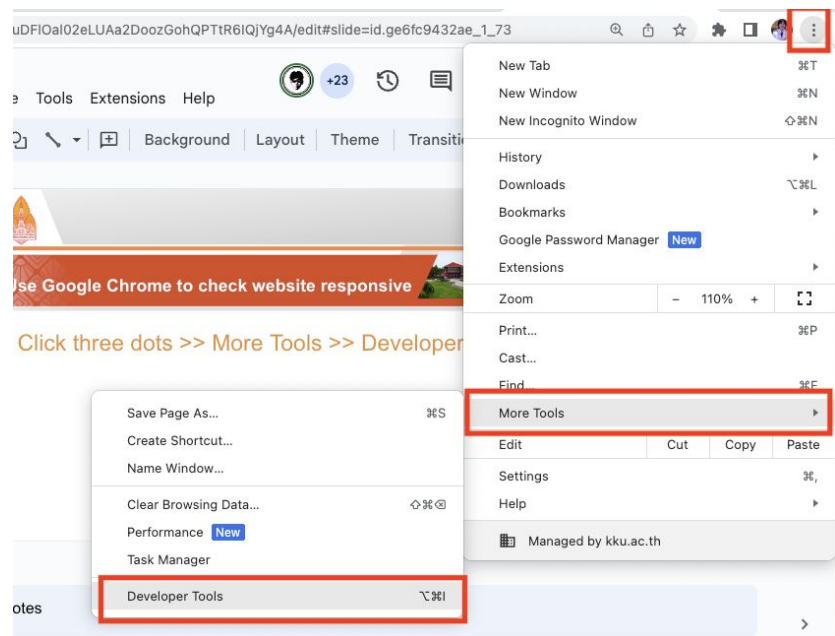
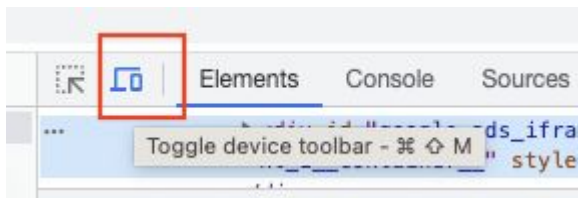
- `50%` → keep horizontally centered
- `20%` → reveal more of the top
- Tune the second value to your photo: `0%–30%` if the subject sits high, `70%–100%` if low.



Use Google Chrome to check website responsive



1. Click three dots >> More Tools >> Developer Tools
2. Press Ctrl + Shift + M

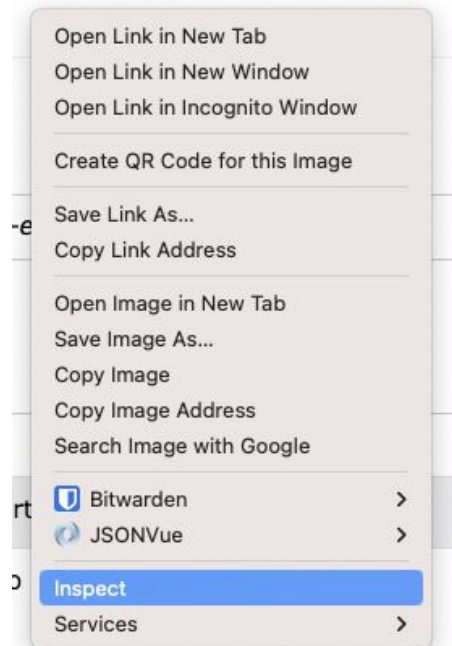
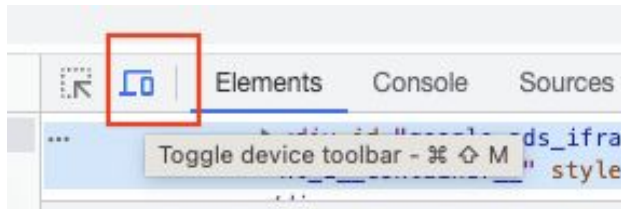




Use Google Chrome to check website responsive



1. Right click on the web page and click Inspect
2. Press Ctrl + Shift + M

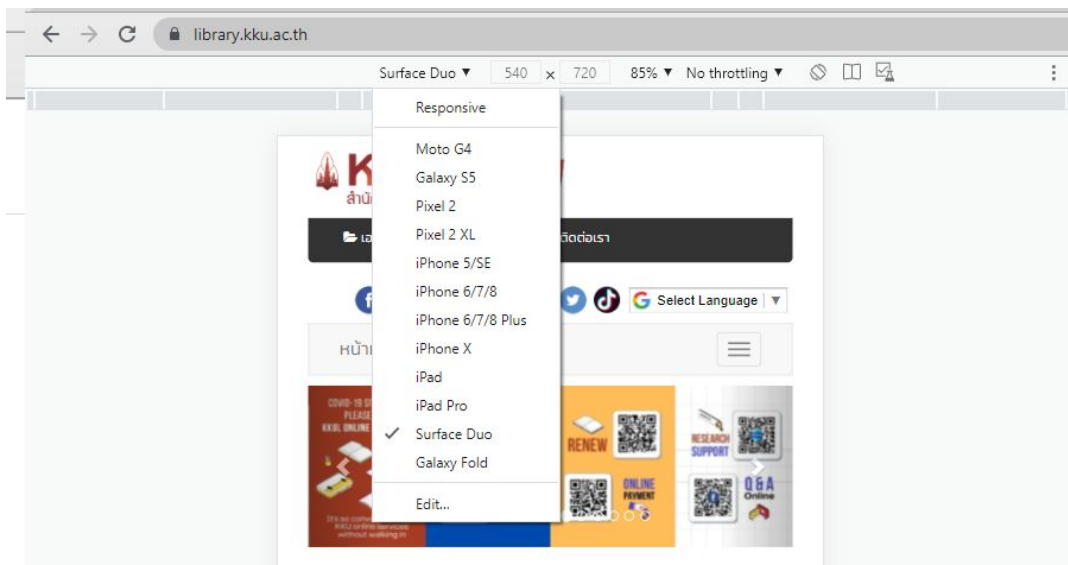




Use Google Chrome to check website responsive



- Select different device screens

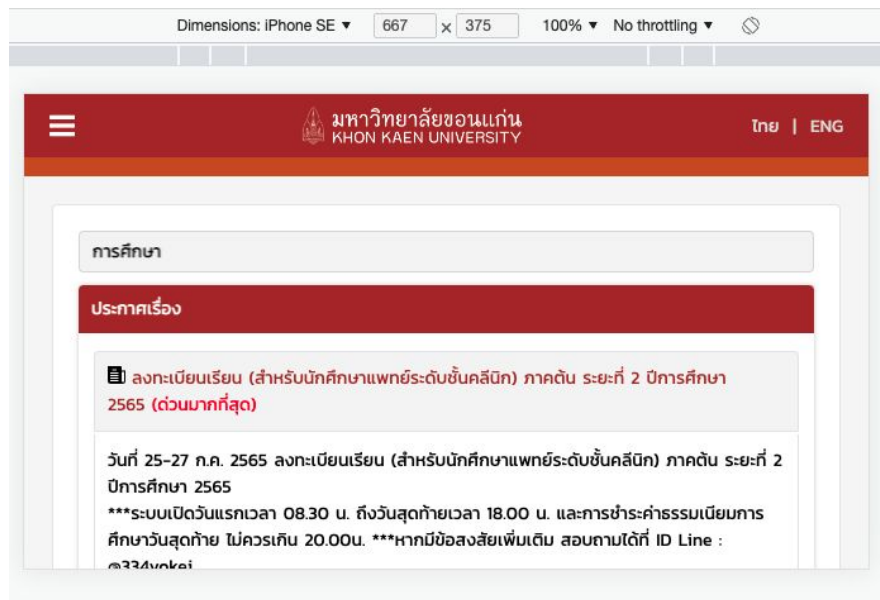
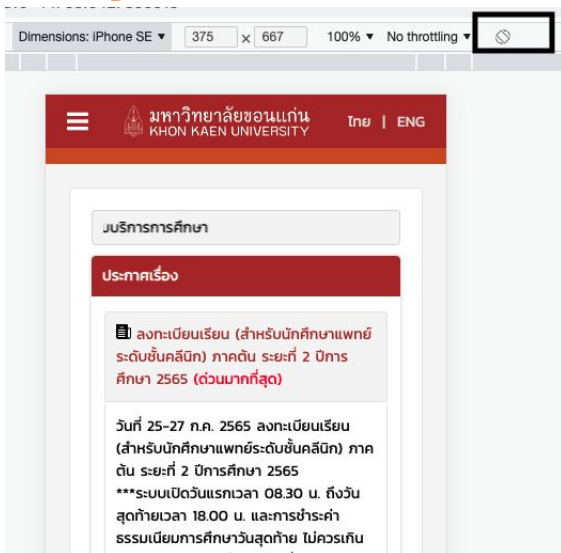




Use Google Chrome to check website responsive



- Change screen orientation





Agenda



- Images
- **CSS Flexbox**
- Alternatives to Responsive Web Design



CSS Flexbox



- The flexbox module is short for a flexible box which is quite similar to the CSS grids
- Flexbox is a one-dimensional module whereas CSS Grids is a two-dimensional module
- Flex-direction has four attributes
 - Row, row-reverse, column, column-reverse
- To create a flex, create a container in which we will design the flexbox
 - `display:flex`



Why Using Flexbox



- Flexbox changes the basic layout behavior. In real-world applications
- Flexbox provides much more control over alignment, distribution of space, and responsive design, making it a powerful tool for creating flexible and responsive layouts.



Sample Usage of Flexbox



responsive > <> flexbox-comparison.html > html > head

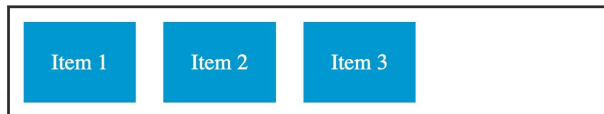
```
21 .flex-container {
22   display: flex;
23 }
24 </style>
25 </head>
26 <body>
27   <h2>Without display: flex</h2>
28   <div class="container">
29     <div class="item">Item 1</div>
30     <div class="item">Item 2</div>
31     <div class="item">Item 3</div>
32   </div>
33
34   <h2>With display: flex</h2>
35   <div class="container flex-container">
36     <div class="item">Item 1</div>
37     <div class="item">Item 2</div>
38     <div class="item">Item 3</div>
39   </div>
40 </body>
```

← → ↺ http://127.0.0.1:5... 🔍 ☆ 📁 | 📄 📥 👤 ⋮

Without display: flex



With display: flex





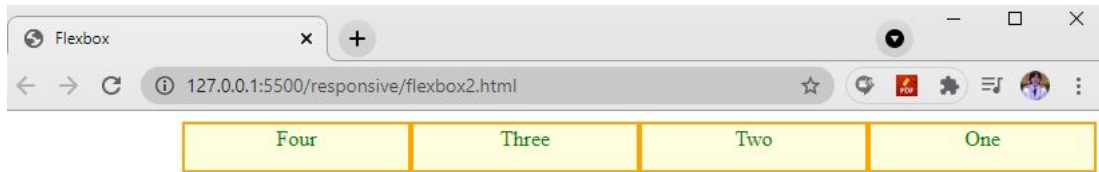
Boxes with flexbox reverse



flexbox2.html U X

responsive > flexbox2.html > head > style > .flexbox

```
1 <!DOCTYPE HTML>
2 <head>
3 <title>Flexbox</title>
4 <meta name="viewport" content="width=device-width, initial-scale=1.0">
5 <style>
6 .flexbox {
7     display: flex;
8     flex-direction: row-reverse;
9 }
10
11 .box {
12     background-color: lightyellow;
13     border: 2px solid orange;
14     padding: 2px;
15     width: 20%;
16     height: 30px;
17 }
18
```



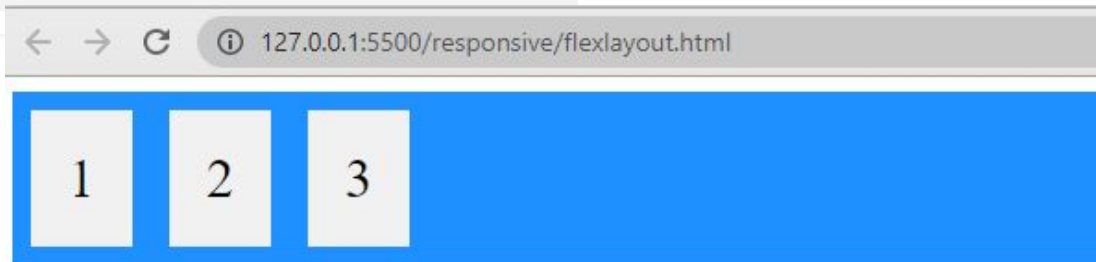


Sample Flexible Layout



flexlayout.html U •
responsive > flexlayout.html > html > body > p > em

```
5 .flex-container {  
6   display: flex;  
7   background-color: #DodgerBlue;  
8 }  
9 .flex-container > div {  
10  background-color: #f1f1f1;  
11  margin: 10px;  
12  padding: 20px;  
13  font-size: 30px;  
14 }  
15 </style>  
16 </head>  
17 <body>  
18 <div class="flex-container">  
19   <div>1</div>  
20   <div>2</div>  
21   <div>3</div>  
22 </div>  
23 <p>A Flexible Layout must have a parent element with the display property set to  
24   flex</p>  
25 <p>Direct child elements(s) of the flexible container automatically becomes flexible items.</p>
```



A Flexible Layout must have a parent element with the *display* property set to *flex*.

Direct child elements(s) of the flexible container automatically becomes flexible items.



The flex-wrap Property



flexwrap.html U

responsive > > flexwrap.html > html > head > style > .flex-container

```
4 <style>
5 .flex-container {
6   display: flex;
7   flex-wrap: wrap;
8   background-color: DodgerBlue;
9 }
10 .flex-container > div {
11   background-color: #f1f1f1;
12   width: 100px;
13   margin: 10px;
14   text-align: center;
15   line-height: 75px;
16   font-size: 30px;
17 }
18 </style></head>
19 <body>
20 <h1>The flex-wrap Property</h1>
21 <p>The "flex-wrap: wrap;" specifies that the flex items will wrap
22 <div class="flex-container">
23   <div>1</div><div>2</div><div>3</div>
24   <div>4</div><div>5</div><div>6</div>
```

127.0.0.1:5500/responsive/flexwr

127.0.0.1:5500/responsive/flexwrap.html

The flex-wrap Property

The "flex-wrap: wrap;" specifies that the flex items will wrap if necessary:



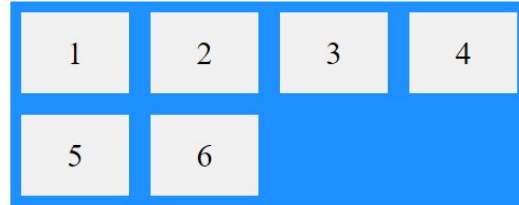
Try resizing the browser window.

127.0.0.1:5500/responsive/flexwr

127.0.0.1:5500/respon...

The flex-wrap Property

The "flex-wrap: wrap;" specifies that the flex items will wrap if necessary:



Try resizing the browser window.



The gap Property in Flexbox



- **gap** sets the space between flex items — it works in Flexbox exactly as it does in Grid
- Applied to the flex **container**, not the items: `.gallery { display: flex; flex-wrap: wrap; gap: 10px; }`
- **gap** adds space **only between** items, never outside them — no leading or trailing gap at the row edges
- With `flex-wrap: wrap`, gaps count toward the line width, so they decide when an item wraps



How many items fit on one row



n items fit when $n \times \text{flex-basis} + (n - 1) \times \text{gap} \leq 100\%$ (the container width)

- Example: `flex-basis: 45%`, `gap: 10px`
→ two tiles = 90% + one 10px gap → fits
- So make `flex-basis` **slightly smaller** than the exact fraction to leave room for the gaps, then let `flex-grow: 1` expand the items to fill the row
- Exact alternative: `flex-basis: calc(50% - 5px)` for 2 per row (two tiles share one gap)



Responsive Flexbox



flexresponsive1.html U x
responsive > <> flexresponsive1.html > html > head > style > .file

```
4 <style>
5 * {
6   box-sizing: border-box;
7 }
8
9 .flex-container {
10  display: flex;
11  flex-direction: row;
12  font-size: 30px;
13  text-align: center;
14 }
15
16 .flex-item-left {
17   background-color: #f1f1f1;
18   padding: 10px;
19   flex: 50%;
20 }
```

flexresponsive1.html U • Settings

responsive > <> flexresponsive1.html > html > body > p

```
22 .flex-item-right {
23   background-color: dodgerblue;
24   padding: 10px;
25   flex: 50%;
26 }
27 /* Responsive layout - makes a one column-layout instead of two-column layout */
28 @media (max-width: 800px) {
29   .flex-container {
30     flex-direction: column;
31   }
32 }
33 </style></head><body>
34 <h1>Responsive Flexbox</h1>
35 <p>The "flex-direction: row;" stacks the flex items horizontally (from left to right).</p>
36 <p>The "flex-direction: column;" stacks the flex items vertically (from top to bottom).</p>
37 <p><b>Resize the browser window to see that the direction changes when the
38 screen size is 800px wide or smaller.</b></p>
39 <div class="flex-container">
40   <div class="flex-item-left">1</div>
41   <div class="flex-item-right">2</div>
42   <div class="flex-item-left">3</div>
43   <div class="flex-item-right">4</div>
```



Responsive Flexbox Output



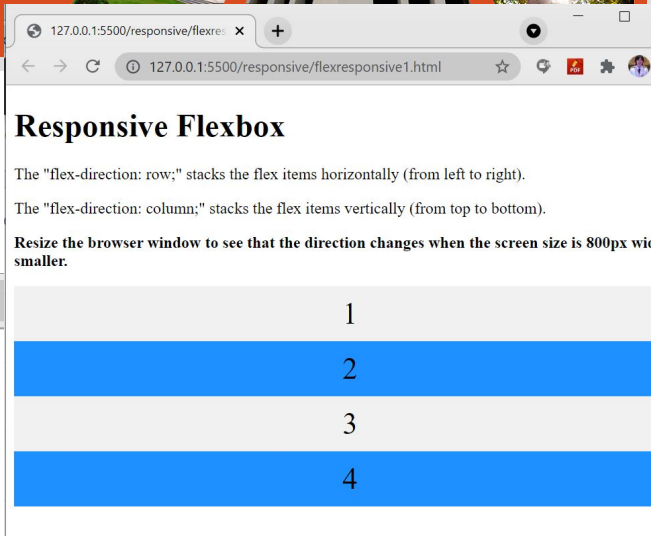
← → ↻ ⓘ 127.0.0.1:5500/responsive/flexresponsive1.html

Responsive Flexbox

The "flex-direction: row;" stacks the flex items horizontally (from left to right).

The "flex-direction: column;" stacks the flex items vertically (from top to bottom).

Resize the browser window to see that the direction changes when the screen size is 800px wide or smaller.

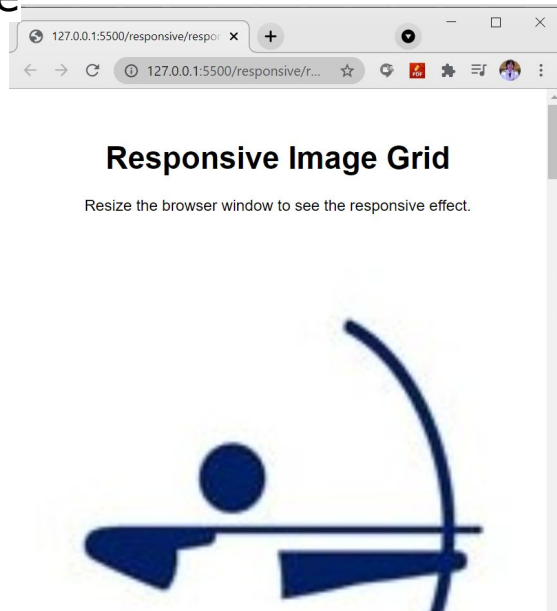




Responsive Image Gallery using Flexbox



- Use flexbox to create a responsive image gallery that varies between two or full-width images, depending on screen size.





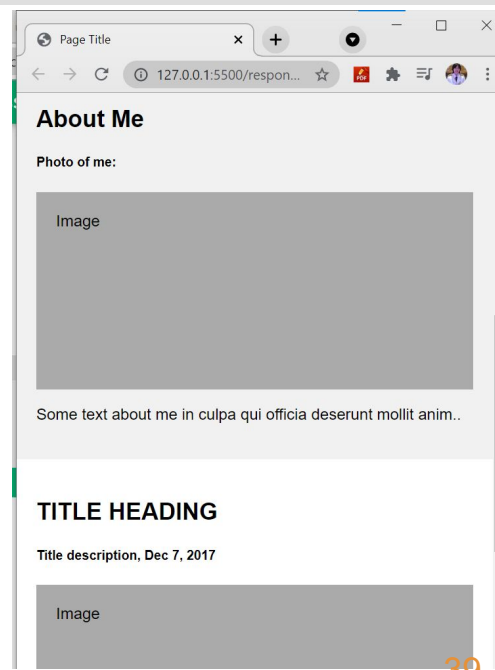
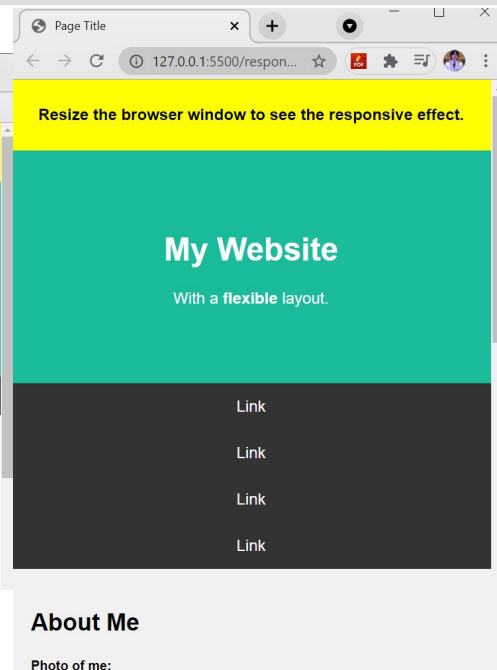
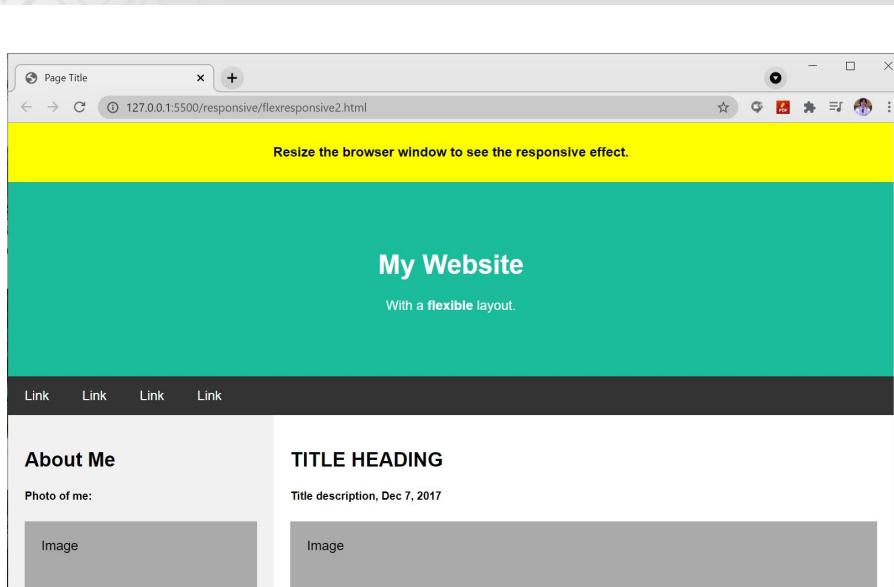
Responsive Image Gallery using Flexbox



```
responsive > responsiveimages.html U x
responsive > responsiveimages.html > html > head > style > .row
1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <style>
6     * {
7       box-sizing: border-box;
8     }
9
10    body {
11      margin: 0;
12      font-family: Arial;
13    }
14
15    .header {
16      text-align: center;
17      padding: 32px;
18    }
19
20    .row {
21      display: flex;
22      flex-wrap: wrap;
23      padding: 0 4px;
24
25    responsive > responsiveimages.html U •
26    responsive > responsiveimages.html > html > head > style > img
27    img {width: 100%;}
28    /* Create two equal columns that sits next to each other */
29    .column {
30      flex: 50%;
31      max-width: 50%;
32      padding: 0 4px;
33    }
34    .column img {
35      margin-top: 8px;
36      vertical-align: middle;
37    }
38    /* Responsive layout - makes a two column-layout */
39    @media (max-width: 800px) {
40      .column {
41        flex: 50%;
42        max-width: 50%;
43      }
44    }
45    /* Responsive layout - makes the stack on top of each other */
46    @media (max-width: 600px) {
47      .column {
48        flex: 100%;
49        max-width: 100%;
50
51    responsive > responsiveimages.html > html > body
52    <body>
53    <!-- Header -->
54    <div class="header">
55      <h1>Responsive Image Grid</h1>
56      <p>Resize the browser window to see the responsive effect.</p>
57    </div>
58
59    <div class="row">
60      <div class="column">
61        
62        
63        
64        
65      </div>
66      <div class="column">
67        
68        
69        
70        
71      </div>
72    </div>
73  </body>
74  </body>
```



Responsive Website using Flexbox





Responsive Website using Flexbox



flexresponsive2.html U X

responsive > flexresponsive2.html > html > head > style > body

```
4 <head>
5 <title>Page Title</title>
6 <meta charset="UTF-8">
7 <meta name="viewport" content="width=device-width, initial-scale=1">
8 <style>
9   * {
10     box-sizing: border-box;
11   }
12
13   /* Style the body */
14   body {
15     font-family: Arial;
16     margin: 0;
17   }
18
19   /* Header/logo Title */
20   .header {
21     padding: 60px;
22     text-align: center;
23     background: #1abc9c;
24     color: white;
25   }
```

flexresponsive2.html U X

responsive > flexresponsive2.html > html > head > style

```
27 /* Style the top navigation bar */
28 .navbar {
29   display: flex;
30   background-color: #333;
31 }
32
33 /* Style the navigation bar links */
34 .navbar a {
35   color: white;
36   padding: 14px 20px;
37   text-decoration: none;
38   text-align: center;
39 }
40
41 /* Change color on hover */
42 .navbar a:hover {
43   background-color: #ddd;
44   color: black;
45 }
46
47 /* Column container */
48 .row {
49   display: flex;
50   flex-wrap: wrap;
```



Responsive Website using Flexbox



flexresponsive2.html U •

responsive > flexresponsive2.html > html > head > style > .fakeimg

```
53  /* Create two unequal columns that sits next to each
54  /* Sidebar/left column */
55  .side {
56    flex: 30%;
57    background-color: #f1f1f1;
58    padding: 20px;
59  }
60  /* Main column */
61  .main {
62    flex: 70%;
63    background-color: white;
64    padding: 20px;
65  }
66  /* Fake image, just for this example */
67  .fakeimg {
68    background-color: #aaa;
69    width: 100%;
70    padding: 20px;
71  }
72  /* Footer */
73  .footer {
74    padding: 20px;
75    text-align: center;
76    background: #ddd;
```

flexresponsive2.html U •

responsive > flexresponsive2.html > html > head > style > {} @media screen and (max-width: 700px)

```
79  /* Responsive layout - when the screen is less than 700px wide, make the two columns s
80  @media screen and (max-width: 700px) {
81    .row,
82    .navbar {
83      flex-direction: column;
84    }
85  }
86  </style>
87  </head>
88
89  <body>
90
91  <!-- Note -->
92  <div style="background: yellow;padding:5px">
93    <h4 style="text-align:center">Resize the browser window to see the responsive effect.<
94  </div>
95
96  <!-- Header -->
97  <div class="header">
98    <h1>My Website</h1>
99    <p>With a <b>flexible</b> layout.</p>
100  </div>
```



Responsive Website using Flexbox



responsive > > flexresponsive2.html > html > body > div.row > div.main > br

```
103 <div class="navbar">
104   <a href="#">Link</a>
105   <a href="#">Link</a>
106   <a href="#">Link</a>
107   <a href="#">Link</a>
108 </div>
109 <!-- The flexible grid (content) -->
110 <div class="row">
111   <div class="side">
112     <h2>About Me</h2>
113     <h5>Photo of me:</h5>
114     <div class="fakeimg" style="height:200px;">Image</div>
115     <p>Some text about me in culpa qui officia deserunt mollit anim..</p>
116   </div>
117   <div class="main">
118     <h2>TITLE HEADING</h2>
119     <h5>Title description, Dec 7, 2017</h5>
120     <div class="fakeimg" style="height:200px;">Image</div>
121     <p>Some text..</p>
122     <p>Sunt in culpa qui officia deserunt mollit anim id est laborum consectetur adipisci
123     tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis r
124   </p><br/>
125   </div>
126 </div>
```



Properties for Flexbox: flex-grow



- Determines how much a flex item will grow relative to other items
- Default value is 0 (doesn't grow)
- Higher values make items grow more
- Example: flex-grow: 2; grows twice as much as flex-grow: 1;



Properties for Flexbox: flex-shrink



- Specifies how much a flex item will shrink relative to others
- Default value is 1 (shrinks evenly with other items)
- Higher values make items shrink more
- Example: `flex-shrink: 0;` prevents item from shrinking



Properties for Flexbox: flex-basis



- Sets the initial main size of a flex item
- Can be a length (e.g., px, em) or a percentage
- Default value is auto (uses item's content size)
- Example: flex-basis: 200px; sets initial width to 200px



Sample Code of Flexbox Properties



```
/* Flex-grow example */
.flex-grow-container .flex-item:nth-child(2) {
  flex-grow: 2;
}

/* Flex-shrink example */
.flex-shrink-container {
  width: 500px;
}
.flex-shrink-container .flex-item {
  flex-basis: 200px;
}
.flex-shrink-container .flex-item:nth-child(2) {
  flex-shrink: 3;
}

/* Flex-basis example */
.flex-basis-container .flex-item {
  flex-basis: 100px;
}
.flex-basis-container .flex-item:nth-child(2) {
  flex-basis: 200px;
}
```

Flex-grow Example

Flex-grow: 1 (default)

Flex-grow: 2

Flex-grow: 1 (default)

Flex-shrink Example

Flex-shrink: 1 (default)

Flex-shrink:
3

Flex-shrink: 1 (default)

Flex-basis Example

Flex-basis:
100px

Flex-basis: 200px

Flex-basis:
100px



The flex Property



The **flex** property is actually a shorthand for three separate properties: **flex: <flex-grow> <flex-shrink> <flex-basis>;**

1. **flex-grow**: This determines how much the item will grow relative to the rest of the flexible items inside the same container. A value of 0 means the item won't grow.
2. **flex-shrink**: This determines how much the item will shrink relative to the rest of the flexible items inside the same container. A value of 1 means the item will shrink evenly with other items.
3. **flex-basis**: This sets the initial main size of a flex item. It can be a length (e.g., px, em) or a percentage.



Examples of the flex Property



`flex: 0 1 calc(50% - 20px);`

- `flex-grow: 0` - The item won't grow beyond its flex-basis size
- `flex-shrink: 1` - The item can shrink if necessary
- `flex-basis: calc(50% - 20px)` - The item's initial size is set to 50% of the container width minus 20px (likely accounting for margins or padding)

`flex: 0 1 100%;`

- `flex-grow: 0` - The item won't grow beyond its flex-basis size
- `flex-shrink: 1` - The item can shrink if necessary
- `flex-basis: 100%` - The item's initial size is set to 100% of the container width



Flexbox vs. CSS Grid



- Flexbox: 1D layouts (rows OR columns); Grid: 2D layouts (rows AND columns)
- Flexbox: Content-first design; Grid: Layout-first design
- Flexbox: Better for component-level layouts; Grid: Better for page-level layouts
- Can be used together for complex, responsive designs



Sample Code Flexbox vs. CSS Grid



```
/* Grid container */
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  background-color: #f0f0f0;
  padding: 10px;
}

.grid-item {
  background-color: #e74c3c;
  color: white;
  padding: 20px;
  text-align: center;
}
```

CSS Grid Example

Grid Item 1	Grid Item 2	Grid Item 3
Grid Item 4	Grid Item 5	Grid Item 6



Building a Navigation Bar with Flexbox



- A nav bar is a row of links — put `display: flex` on the `<nav>` to lay them out.
- Mobile-first: base style stacks links vertically with `flex-direction: column`; a `min-width` query switches to a row.
- `flex: 1` on each link makes them share the row equally (grow to fill, equal widths). In a column, links are full-width by default (cross-axis stretch).



Building a Navigation Bar with Flexbox



```
/* mobile-first: stacked, full width */  
nav { display: flex; flex-direction: column; }  
nav a { text-align: center; padding: 12px; text-decoration: none; }  
@media (min-width: 769px) {  
    nav { flex-direction: row; } /* one row on desktop */  
    nav a { flex: 1; } /* equal-width links fill the row */  
}
```



Link States and the LVHA Order



- Links have four states, styled with pseudo-classes: `:link` (unvisited), `:visited`, `:hover`, `:active` (while clicked).
- Order matters: rules of equal specificity — the later one wins. Write them **LVHA**: `:link` → `:visited` → `:hover` → `:active`. Mnemonic: **LoVe / HAt**e.
- Wrong order breaks it: put `:hover` before `:visited` and a visited link won't show its hover style.



Link States and the LVHA Order



```
nav a:link { color: #333; }
```

```
nav a:visited { color: #333; }
```

```
nav a:hover { background: #ddd; } /* pointer is over the  
link */
```

```
nav a:active { background: #bbb; } /* while the link is  
being clicked */
```



Full-Height Sections: the `vh` Unit + Anchor Links



- `vh` = viewport-height unit. `1vh` = 1% of the viewport height, so `100vh` = the full screen height.
- `min-height: 100vh` makes each section at least one screen tall. Use `min-height`, not `height`, so content taller than the screen still shows fully.
- Anchor links jump to a section by `id`: give the section an `id`, link to it with `href="#id"` (the href value, minus `#`, must equal the id).



Full-Height Sections: the vh Unit + Anchor Links



```
<nav>
```

```
  <a href="#home">Home</a>
```

```
  <a href="#about">About</a>
```

```
</nav>
```

```
<section id="home"> ... </section>
```

```
<section id="about"> ... </section>
```

```
section { min-height: 100vh; }
```



Sticky Navigation



- **position: sticky** keeps the nav on screen while scrolling: it flows normally until it hits the offset you set, then sticks.
- **top: 0** is the stick point; **z-index** keeps the nav above section content so sections don't scroll over it.

```
nav {  
    position: sticky;  
    top: 0; /* stick to the top edge */  
    z-index: 10; /* stay above the sections */  
}  
  
section {  
    scroll-margin-top: 70px; } /* ≈ nav height */
```



Agenda



- Images
- CSS Flexbox
- **Alternatives to Responsive Web Design**



Bootstrap



- Bootstrap is a popular, modern front-end/UI development framework
- It will have most of the things you'll need for developing responsive sites and apps
- Bootstrap is well-documented, and this open source project has plenty of coverage in blogs and tutorial sites
- Bootstrap Tutorials
 - [Getting Started with Bootstrap](https://twitter.github.io) (twitter.github.io)
 - [Twitter Bootstrap 101: Introduction](https://webdesign.tutsplus.com) (webdesign.tutsplus.com)



Pros and Cons of Bootstrap



Pros:

- Comprehensive component library
- Extensive documentation
- Large community support
- Responsive grid system

Cons:

- Can be heavy if not optimized
- Websites may look similar (if not customized)
- Learning curve for advanced features



TailwindCSS



- Bootstrap is a component-based framework, which means it comes with prebuilt components and include other utilities for layering displays, spacing
- Tailwind, on the other hand, is a utility-first framework
- Using Tailwind CSS is akin to writing regular CSS
- Unlike Bootstrap, it has no prebuilt components



Pros and Cons of TailwindCSS



Pros:

- Highly customizable
- Smaller file sizes (when purged)
- Rapid prototyping capabilities
- No pre-styled components (unique designs)

Cons:

- Steeper learning curve
- HTML can become cluttered with classes
- Requires additional setup for optimal performance



Foundation



- Foundation is another popular responsive front-end framework
- It has rapid-prototyping capabilities, a responsive grid system and much more
- Foundation Tutorials
 - [Foundation Docs: Getting Started](http://zurb.com) (zurb.com)
 - [A Beginner's Guide to Zurb Foundation](http://designshack.net) (designshack.net)



Pros and Cons of Foundation



Pros:

- Flexible grid system
- Emphasis on semantic markup
- Advanced features like motion UI

Cons:

- Smaller community compared to Bootstrap
- Can be complex for beginners
- Less frequent updates



HTML5 Boilerplate



- HTML5 Boilerplate is a mobile-friendly HTML template, placeholder icons, CSS resets for normalizing/standardizing your stylesheet property values
- HTML5 Boilerplate Tutorials
 - [The Official Guide to HTML5 Boilerplate](http://net.tutsplus.com) (net.tutsplus.com)
 - [HTML5 Quick Start with Boilerplate](http://teamtreehouse.com) (teamtreehouse.com)



Pros and Cons of HTML5 Boilerplate



Pros:

- Lightweight starting point
- Includes best practices and optimal defaults
- Great for custom projects

Cons:

- Not a full framework (minimal styling included)
- Requires more manual work for responsiveness
- Less suitable for rapid prototyping



Conclusions



- **Responsive Images:** Utilize techniques like **width**, **max-width**, **object-fit**, and **object-position** to ensure images adapt well to different screen sizes.
- **CSS Flexbox:** A powerful tool for creating flexible, one-dimensional layouts that automatically adjust to different screen sizes and content amounts.
- **Framework Alternatives:** Consider options like Bootstrap, TailwindCSS, Foundation, or HTML5 Boilerplate based on project needs and development preferences.
- **Tools and Testing:** Use browser developer tools to test and refine responsive designs across various device sizes and orientations.



References



- https://www.w3schools.com/css/css_rwd_intro.asp
- <https://blog.hubspot.com/website/w3-css-vs-bootstrap>
- <https://css-tricks.com/all-about-floats/>
- <https://www.lambdatest.com/blog/css-media-queries-for-responsive-design/>
- <https://www.lambdatest.com/blog/responsive-web-design-all-you-need-to-know/>
- [https://www.lambdatest.com/blog/how-to-make-responsive-images-with-css-html-wordpre ss-more/](https://www.lambdatest.com/blog/how-to-make-responsive-images-with-css-html-wordpress-more/)
- <https://www.webfx.com/blog/web-design/html5-frameworks/>
- <https://www.lambdatest.com/blog/how-to-use-css-breakpoints-for-responsive-design/>
- https://www.w3schools.com/css/css3_flexbox_responsive.asp
- <https://www.olympic-museum.de/pictograms/olympic-games-pictograms-2021.php>
- <https://blog.logrocket.com/tailwind-css-vs-bootstrap-ui-kits/>
- <https://claude.ai>

